ChromaScope

Análisis Distribuido de Color Dominante en Imagen con Driver de Kernel y LEDs RGB

CE 4303 — Principios de SO
Profesor: Jason Leitón
Entrega: 12 de Junio, 2026

¿Qué hace el sistema?

El cliente carga un **archivo de imagen (PPM)** desde disco, le **aplica un filtro de escala de grises** y guarda el resultado en un nuevo **archivo PPM**. A la imagen original se le calcula el histograma de color. La imagen original se **cifra byte a byte** con ChaCha20 (implementado en C puro) y se **distribuye mediante OpenMPI** al clúster de procesamiento. Las cuatro computadoras físicas procesan fragmentos de la imagen en paralelo usando **MPI_Send y MPI_Recv**, cada una calculando un histograma de color parcial. El resultado consolidado —el histograma completo de la imagen— se muestra físicamente en una matriz de **LEDs RGB** controlada por un **driver de kernel Linux** desarrollado a medida **vía GPIO en una Raspberry Pi**.

① I/O — Driver de Kernel

Módulo .ko en C que expone /dev/chromascope con operaciones ioctl para controlar una matriz de LEDs RGB (PWM) y un servomotor SG90 vía GPIO. Verificable con lsmod. Sin drivers del sistema ni comerciales. Como punto extra, se desarrolla un driver en C para una pantalla TFT LCD de 1.8 pulgadas donde se visualiza el histograma de color de forma gráfica.

② Hardware — Prototipo Físico

Raspberry Pi con matriz de LEDs RGB de ánodo común + servomotor SG90. La matriz de LEDs representa visualmente el histograma de color dominante de la imagen. El servomotor hace un barrido de 0°→90° como señal de finalización (Opcional). Como punto extra, se incorpora un ESP32-C6 para visualización adicional del histograma en pantalla TFT LCD de 1.8 pulgadas.

③ Distribución — Clúster MPI

4 PCs físicas interconectadas por LAN. La distribución de franjas de imagen se realiza mediante comunicación punto a punto con MPI_Send y MPI_Recv; cada nodo calcula un histograma HSV parcial; MPI_Reduce suma los resultados en el maestro. Sin compartir archivos. Toda la comunicación entre nodos se realiza exclusivamente a través de OpenMPI, sin uso de TC.

④ Procesamiento de Imagen — Filtro y Histograma

Antes de la distribución, el sistema aplica un filtro de escala de grises a la imagen original y guarda el resultado en un nuevo archivo PPM. A partir de la imagen original (sin filtro) se extrae el histograma de color HSV, el cual es la entrada del pipeline de procesamiento distribuido.

⑤ Seguridad — Cifrado ChaCha20

Toda la comunicación distribuida va cifrada con ChaCha20 implementado en C puro (RFC 7539, ~200 líneas). Clave de 256 bits + nonce de 96 bits por sesión. Sin OpenSSL ni libsodium.

Flujo end-to-end

1. Cliente aplica filtro de escala de grises a la imagen PPM → guarda imagen filtrada en archivo

2. Cliente cifra imagen PPM original con ChaCha20

3. Nodo 0 recibe la imagen cifrada vía OpenMPI, descifra y divide en 4 franjas horizontales

4. MPI_Send → Nodos 1, 2, 3 y 4 reciben su franja; MPI_Recv confirma recepción; cada nodo calcula histograma HSV parcial

5. MPI_Reduce suma histogramas → Nodo 0 determina histograma completo

6. Biblioteca (.a) → driver /dev/chromascope → GPIO → Matriz de LEDs + servomotor

7. (Opcional) Driver TFT LCD → pantalla 1.8" en ESP32-C6 muestra histograma gráficamente

Stack técnico

Lenguaje: C (C11), GCC, sin VMs

Distribución: OpenMPI 4.x, SSH sin contraseña, MPI_Send/MPI_Recv

Driver: Linux Kernel Module API, GPIO

Cifrado: ChaCha20 RFC 7539 (C puro)

Imagen: Formato PPM, filtro escala de grises, histograma HSV

Hardware: Raspberry Pi + Matriz LEDs RGB + SG90

Hardware extra: ESP32-C6 + Pantalla TFT LCD 1.8"